Needle in the haystack: LLMs for vulnerability research

09 Mar, 2026

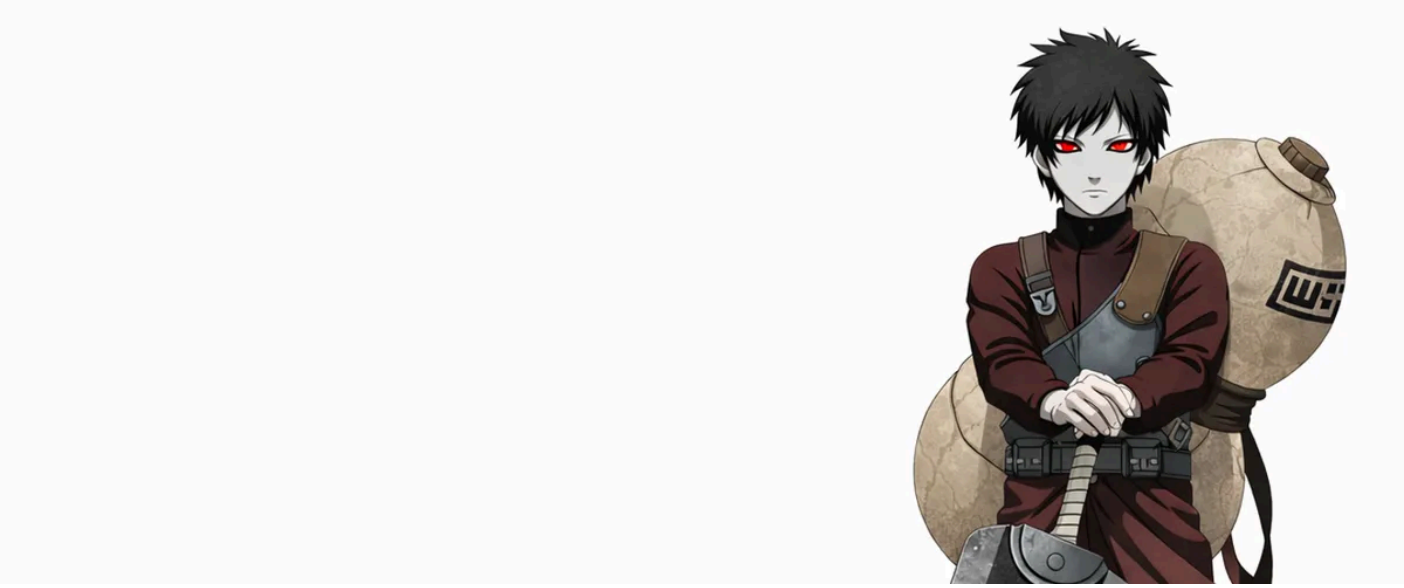


Table of Contents

- Intro Lore
- Why "Find All The Vulnerabilities" does not work
- Minimal Scaffolding That Actually Helps
- Case Study: Claude Opus 4.6 and Firefox
  - What Anthropic Actually Did
- My Own Methodology
  - The Approach
    - Parse Server
    - HonoJS
    - ElysiaJS
    - harden-runner
    - BullFrog

        - Better-Hub
    - Vulnerabilities Found
    - Why This Worked
- The Sweet Spot
- Prompt Injection
- References

Note: Initially, the idea was to write a single article covering the entire methodology and all the technical details behind the techniques I use, including AI-powered differential and grammar-based fuzzing, automated harness generation, and related workflows. However, I realized that packing everything into one article would make it unnecessarily dense and difficult to follow.

Instead, this post serves as the first installment, presenting a high-level overview of the methodology and the key ideas behind the approach. Future posts will dive deeper into the technical details and implementation aspects of each component.

Everything shared here is intended strictly for educational and research purposes. Any misuse or malicious activity carried out using the information discussed is solely the responsibility of the individual performing it.

Intro Lore

I reported a bunch of security issues in the last few weeks. A small portion of these vulnerabilities have now been fixed and disclosed in the form of security advisories. All of these vulnerabilities were found 100% using LLMs without any manual source code review. The projects in which I found these vulnerabilities are pretty well-known and widely used. Some of these projects include big names like Parse Server, HonoJS, ElysiaJS, Harden Runner, and around a dozen more big names.

I feel this proves that agentic CLIs and TUIs like OpenAI Codex can no doubt help you find serious vulnerabilities. But how do we actually use these tools to uncover obscure vulnerabilities? Based on my tests and after sending thousands and thousands of prompts in order to discover the vulnerabilities, I came to some conclusions. They might not be theoretically accurate, but these are some of the most pragmatic conclusions that I arrived at.

I found that some of the fastest ways to miss important vulnerabilities are:

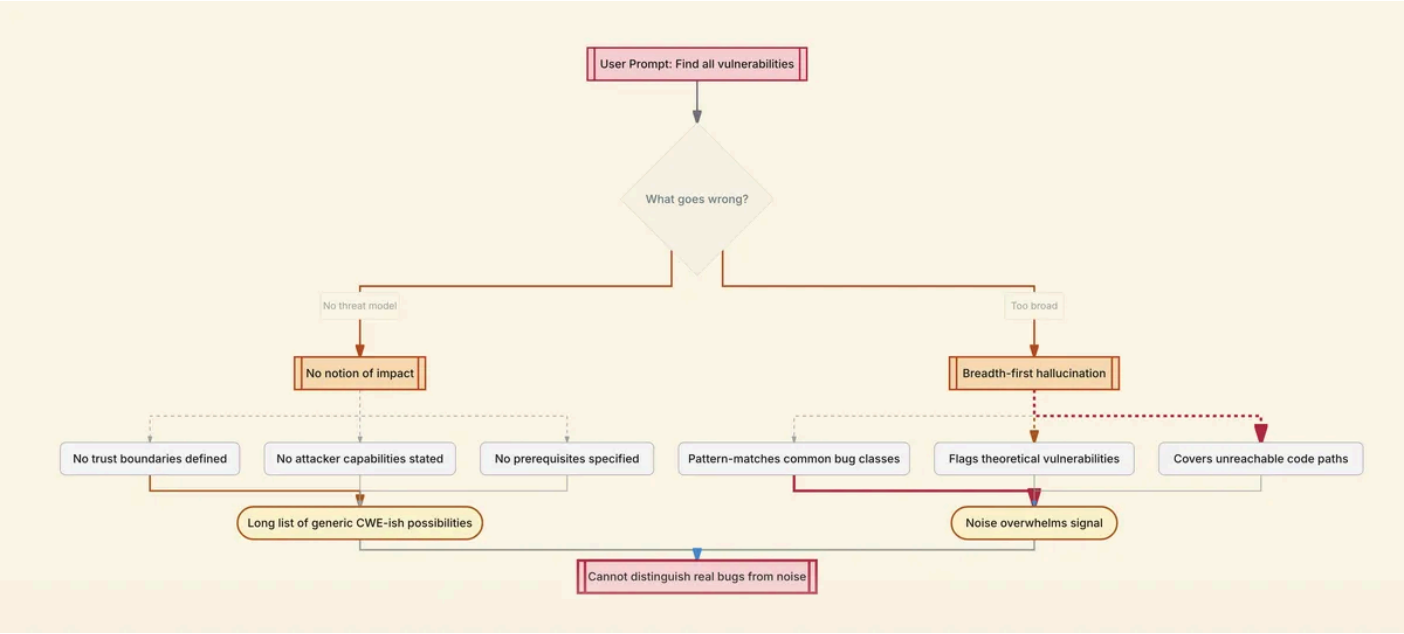
1. Over-scaffolding the security audit by chaining prompts
2. Bloated AGENT.md/SKILLS.md files

3. Giving too much context in the form of documents or pre-planning every step of the process
4. Trying to orchestrate way too much

But that sounds counterintuitive, no? Any sane person will think guidance should mean better results, right? But long-context systems have a very real and well-researched problem. As you stuff more tokens into the context window, the model's reliability at picking the right details degrades. Recent work explicitly describes this as **context rot** where performance becomes increasingly unreliable as context length grows, even when the added content is technically relevant. Security auditing is a worst-case environment for this. The "needle" is often a single subtle invariant violation buried among thousands of legitimate lines. Based on my tests, I discovered that, in many cases, models exhibit primacy/recency behavior, doing better when the relevant "needle" is near the beginning or end of context and worse when it's buried in the middle. That is the needle-in-the-haystack problem in its purest form.

So what should we do? Should we get rid of our AGENTS.md file and run the LLM wild with no scaffolding? That leads to some even bigger problems, but that's a topic for some other time. For now, what I have nailed down based on my tests/experience driven from finding over a dozen CVEs in popular open-source projects is that the trick is minimal persistent scaffolding, maximal targeted exploration and verification, and a workflow that keeps the model's attention anchored to what matters.

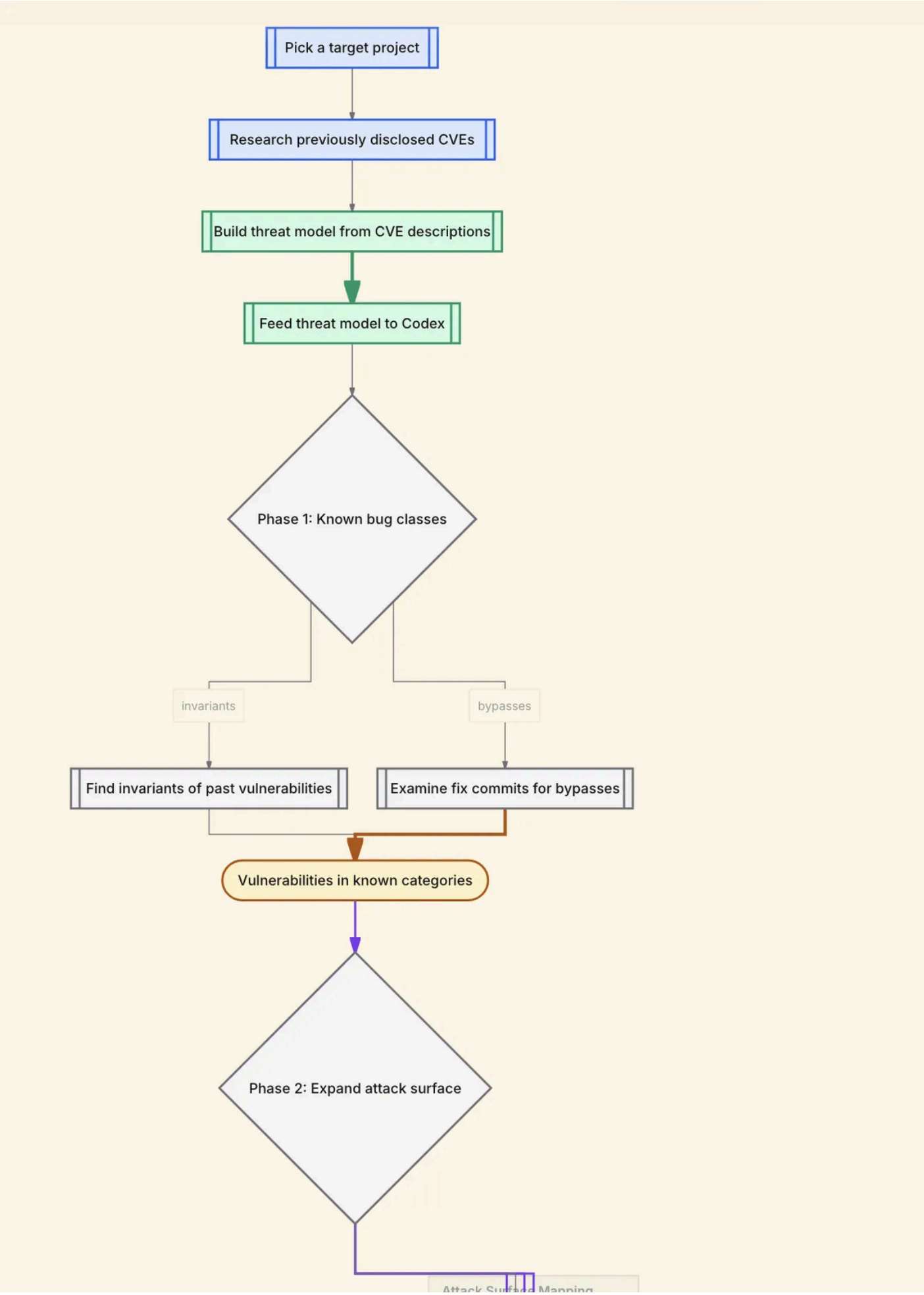
Why "Find All The Vulnerabilities" does not work

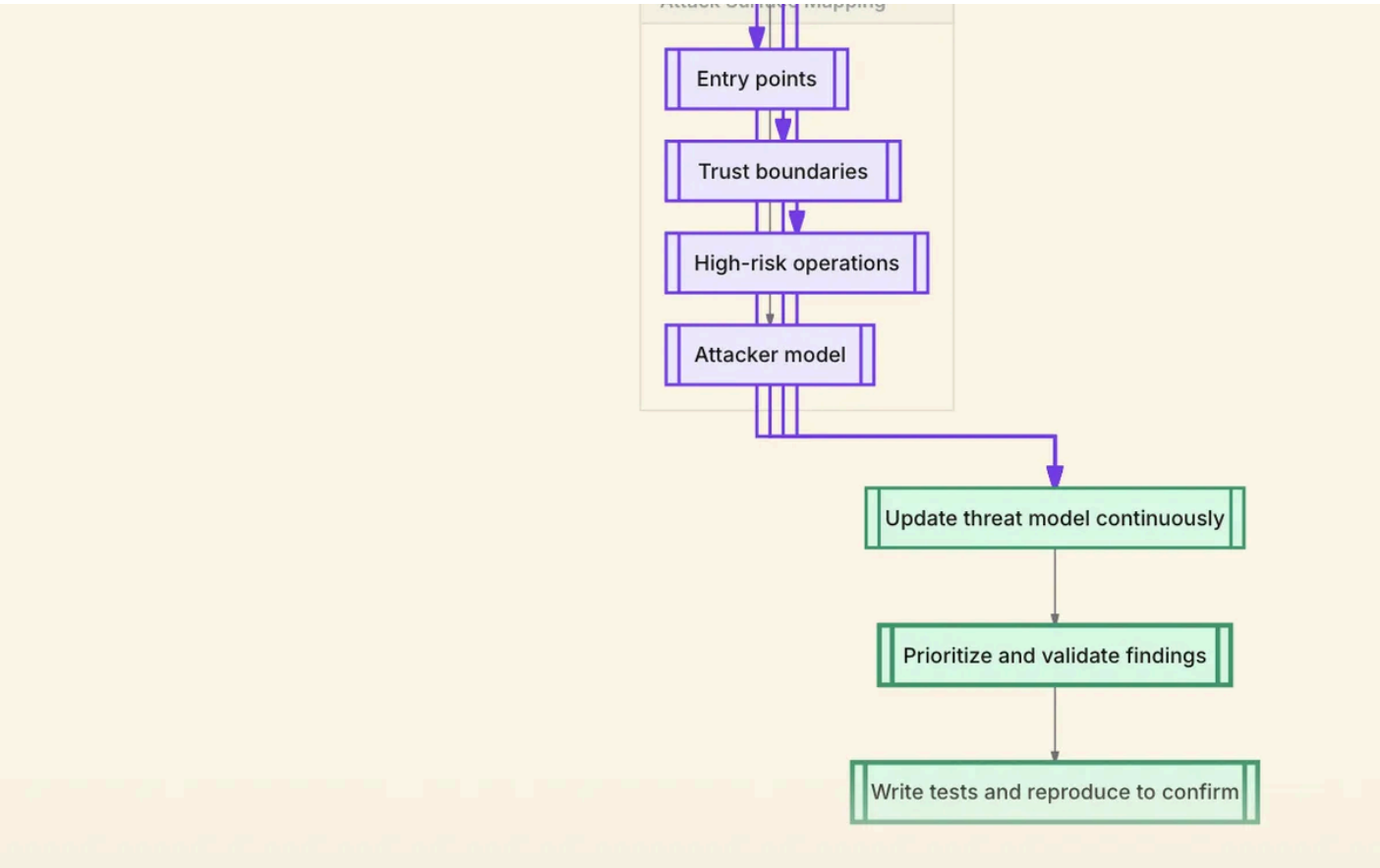


Let's say you have a large folder containing monolithic source code or maybe you have cloned a repository from GitHub and you want to find security vulnerabilities in that source code. The first thing you do is initiate Codex and then type in the prompt "find all vulnerabilities in this codebase." Now this particular prompt fails for two predictable reasons.

- When you gave the prompt, you did not specify any threat model. As a result, the LLM has no notion of impact. It can derive some kind of threat model, but generally it does not, or does so poorly, and this is based on my experiments. Without a proper threat model, trust boundaries, attacker capabilities, and prerequisites, the findings that you will be getting will be a long list of generic CWE-ish possibilities with no prioritization. There is not going to be a way for you to distinguish interesting findings from the long list of noise.
- The second reason is that when you gave the prompt, it pushed the model into a breadth-first hallucination. We know that broad prompts invite broad answers. The model will pattern-match to find common bug classes even when they are not possible in your code context. You end up reviewing theoretical vulnerabilities in code paths no attacker could reach.

Minimal Scaffolding That Actually Helps


Pick a target project
Research previously disclosed CVEs
Build threat model from CVE descriptions
Feed threat model to Codex
Phase 1: Known bug classes
invariants
bypasses
Find invariants of past vulnerabilities
Examine fix commits for bypasses
Vulnerabilities in known categories
Phase 2: Expand attack surface



Now that we have seen what giving a vague prompt can lead to, let's try to do things the right way. Before asking an LLM to audit code in order to find vulnerabilities, try to do what human security teams do to identify the threat model. This is something you can generate via LLM as well.

What I usually do is look for the previously disclosed CVEs in that project and based on the descriptions of those CVEs, I prompt the LLM to create a threat model for plausible bug classes based on the CVE descriptions that we have accumulated.

So let's say the previously disclosed CVEs were related to heap overflow, stack overflow, integer overflow, and memory corruption, the LLM will try to build a threat model for these kinds of vulnerabilities because these were previously accepted by the project as positive vulnerabilities.

Now take that threat model doc you just created, feed it to Codex and ask the LLM to find invariants of it, or maybe try to look for the commit that fixed these vulnerabilities and try to find bypasses for that. That is likely to fetch you more vulnerabilities as compared to giving a vague prompt.

In this case, the minimal scaffolding you did was creating a threat model. Other than that, we did not make any skills.md file or agents.md file. You did not try to orchestrate a lot of things. You just created a threat model, gave it to the

LLM, and now the LLM is going to do deep research in the codebase and will try to find vulnerabilities that fall into that threat model.

Now, after you are done with trying to find vulnerabilities that fall into the same category as previously disclosed CVEs, try to identify the entry points such as HTTP routes, RPC handlers, message consumers, CLI entrypoints, and scheduled jobs. Identify the trust boundaries such as browser to server, service to service, plugin to host, and sandbox to privileged. Identify high-risk operations such as deserialization, templating, native bindings, authz checks, and parsing untrusted inputs. And explicitly state the attacker-victim model, for example, you want to find vulnerabilities that can be triggered by a remote unauthenticated user, a remote authenticated low-privileged user, or a cross-tenant user.

This is the kind of small structure that improves signal without bloating the context window. Threat modeling is the ultimate compression algorithm for your security audit.

The important thing, or I could say the only important thing, is to build the system context first. Then create an editable threat model. Keep on extending that threat model as you progress during your security audit. Keep on adding new things and then use that threat model to prioritize findings and eventually validate.

Case Study: Claude Opus 4.6 and Firefox

Anthropic's March 6, 2026 write-up describes a collaboration with Mozilla where Claude Opus 4.6 discovered 22 vulnerabilities in about two weeks, and Mozilla assessed 14 of them as high-severity. Mozilla's own post confirms the result and emphasizes why it worked in practice. The reports came with minimal test cases that made reproduction and fixing fast, and the team expanded the technique beyond the JS engine across the browser.

What Anthropic Actually Did

Anthropic's description is not "we wrote a mega-prompt." It is closer to the following. They started with a focused slice of the codebase, the JavaScript engine, because it is critical and analyzable in isolation. They iterated quickly. Claude found a use-after-free after roughly 20 minutes of exploration, humans validated it, and they filed a Bugzilla report with a candidate patch. They scaled out once the workflow proved itself, ultimately scanning around 6,000 C++ files and submitting 112 unique reports, with most fixes landing in Firefox 148.0 (released February 24, 2026).

Now is this the right approach? Maybe yes, maybe not. The thing is, many of the vulnerabilities that Anthropic must have found were not valid. They had to report vulnerabilities that were exploitable, and in order to find exploitable vectors, they had to go from a potential vulnerability to validating it and then to confirming that it is exploitable. That chain is very expensive. It cost Anthropic approximately $4,000 in API credits. Can we spend that amount of money while auditing code? Maybe not. But are we dealing with the same scale of code as Firefox? Also not.

Anthropic did it with almost no scaffolding at all. But what I am trying to advocate for is to have minimal scaffolding in the form of creating a threat model and describing the trust boundaries first. And this is just about finding vulnerabilities. I'm not really going into evaluating them and eliminating false positives. There are ways and approaches to work towards that, but that's maybe a topic for another blog. For now, what you can do is, if after creating a threat model the LLM is finding some vulnerabilities, you can instruct Codex to run a local instance after building the source or write tests that prove the existence of vulnerabilities. Most of the time it works.

My Own Methodology

Minimal scaffolding works just fine and gives us many more vulnerabilities and even certain footguns and edge behaviors that the vague prompts will never give you. I want to illustrate this with my own recent work which resulted in the discovery of over 30 vulnerabilities across multiple different projects in roughly two months.

The Approach

Every audit started the same way. Pick a thin slice and understand its trust model before asking the LLM to find vulnerabilities in the codebase.

Parse Server

Detailed write-up: Four Vulnerabilities in Parse Server

Parse Server is an open-source backend framework that provides a REST API, real-time queries, push notifications, and cloud functions. It supports multiple authentication mechanisms, including a `readOnlyMasterKey` that the documentation promises will grant master-level reads but deny all writes.

Before prompting the LLM to look for anything, I pulled the previously disclosed CVEs for Parse Server. Past advisories showed a recurring pattern of

authorization enforcement failures, cases where privilege checks existed but were incomplete or inconsistently applied across route handlers. I fed those CVE descriptions to the LLM and asked it to generate a threat model for plausible bug classes based on that history. The model identified authorization boundary enforcement as the dominant risk category, which made sense given Parse Server's architecture of multiple key types with different privilege levels.

That threat model surfaced the `readOnlyMasterKey` as an interesting trust boundary. The claim is simple. One key type should have strictly fewer capabilities than another. I pointed the LLM at this boundary and asked it to explore how the different key types interact with the authorization layer, what assumptions the code makes about privilege separation, and where those assumptions might break down.

The LLM came back with an attack surface map that highlighted a pattern: several route handlers gate access on `isMaster` but never consult `isReadOnly`. That was the signal. I followed up with a narrower prompt asking it to enumerate every handler exhibiting this pattern and trace whether the read-only credential could reach write or state-changing operations through any of them.

Three of the four vulnerabilities (CVE-2026-29182, CVE-2026-30228, CVE-2026-30229) came from that same root cause. Once those were confirmed, I opened a separate slice targeting the social auth adapters with a similarly guided approach. I pointed the LLM at the authentication adapter layer and asked it to explore the token validation flow, what claims are checked, what happens when configuration is partial or missing, and where the validation might silently degrade. The LLM identified the JWT audience validation path as a weak point, and a follow-up prompt confirmed the fourth finding, CVE-2026-30863, an independent JWT audience validation bypass where the adapter silently skipped the `aud` claim check when configuration was incomplete. Different slice, same approach.

HonoJS

Detailed write-up: HonoJS JWT/JWKS Algorithm Confusion

HonoJS is a lightweight, high-performance web framework for JavaScript and TypeScript that runs across multiple runtimes including Cloudflare Workers, Deno, Bun, and Node.js. It ships built-in middleware for JWT and JWKS-based authentication.

I started by reviewing Hono's past security advisories and any previously disclosed issues in its authentication middleware. The CVE history (even though there were very few disclosed CVEs), combined with the general pattern of JWT implementation mistakes across the ecosystem (in other projects), pointed the LLM

toward algorithm handling as a high-risk area when I asked it to build a threat model. The model flagged algorithm confusion and default fallback behavior as the most plausible bug classes, which gave me a clear slice. The JWT and JWKS verification paths.

With that threat model in hand, I directed the LLM to explore the algorithm selection logic in the JWT middleware. Rather than asking about a specific flaw, I asked it to walk through what happens when developers don't configure things perfectly, what defaults kick in, what fallback paths exist, and how the middleware decides which algorithm to trust. The goal was to have the LLM map out the decision tree for algorithm selection and flag any branches where the middleware might be making unsafe assumptions.

The LLM surfaced two concerning patterns in its analysis. First, it identified a fallback to HS256 when no algorithm is explicitly pinned. Second, it flagged the JWKS middleware's behavior of deferring to the token's `header.alg` value when the JWK key object lacks an `alg` field. I followed up on each with targeted prompts asking the LLM to trace the exact conditions under which an attacker could exploit these fallbacks.

Two algorithm confusion issues fell out. CVE-2026-22817 was the JWT middleware defaulting to HS256 when no algorithm was pinned, allowing an attacker to sign tokens with the public key as an HMAC secret. CVE-2026-22818 was the JWKS middleware falling back to the untrusted `header.alg` value when the JWK lacked an `alg` field, letting an attacker dictate which algorithm the server used for verification.

ElysiaJS

Detailed write-up: ElysiaJS Cookie Signature Validation Bypass

ElysiaJS is a TypeScript web framework built for Bun, emphasizing type safety and developer ergonomics. It includes built-in cookie handling with signature-based integrity verification and support for secrets rotation.

The threat model generation followed the same pattern. I looked at ElysiaJS's documentation. When I fed that context to the LLM and asked it to identify plausible bug classes, it flagged signature verification logic as a high-risk area, particularly the secrets rotation path, where multiple signing keys may be valid simultaneously and the verification logic has to correctly reject cookies that match none of them.

That gave me a narrow slice. I pointed the LLM at the cookie signing and verification layer and asked it to reason about the state management during

verification, how does the code track whether a signature has been successfully validated, what happens when it iterates through multiple rotated secrets and none of them match, and are there any initialization assumptions that could cause the logic to silently accept an invalid signature?

The LLM identified the `decoded` status variable as suspicious and flagged its initialization. Following up on that signal, I asked it to trace the control flow when no secret produces a matching signature. That confirmed the bug: a single boolean initialization error, `let decoded = true` instead of `let decoded = false`, meant the signature validation check could never fail when using secrets rotation. The CVE is still pending.

---

harden-runner

Detailed write-up: Bypassing Outbound Connections Detection in harden-runner

harden-runner is a security tool by StepSecurity for GitHub Actions that monitors outbound network connections from CI/CD runners by instrumenting syscalls to detect unauthorized egress. It operates in two modes. Audit mode logs connections, and block mode actively prevents them.

I reviewed harden-runner's previous advisories and its documented security model. The tool's entire value proposition rests on complete visibility into outbound network activity, so the threat model I asked the LLM to generate was centered on a single question. "Can an attacker with code execution on a GitHub Actions runner exfiltrate data past the egress controls?" The LLM identified syscall coverage gaps as the most likely bypass class, given that the tool works by hooking specific system calls and any call outside the monitored set would be invisible.

With that threat model, I pointed the LLM at the syscall monitoring layer and asked it to explore the coverage surface. What families of syscalls are being hooked, what are the different ways a process can send data over the network on Linux, and are there any gaps between the two? The idea was to have the LLM enumerate the full set of network-related syscalls and then compare that against what harden-runner actually instruments.

The LLM came back with a gap analysis that flagged UDP send-family syscalls as potentially unmonitored. I followed up asking it to verify specifically which of `sendto`, `sendmsg`, and `sendmmsg` were covered. The bypass was exactly what the threat model predicted: those syscalls fell outside the monitoring scope in audit mode (CVE-2026-25598).

BullFrog

Detailed write-ups: Bypassing egress filtering in BullFrog GitHub Action, sudo restriction bypass in BullFrog GitHub Action, Bypassing egress filtering in BullFrog using shared IP

BullFrog is another security tool for GitHub Actions that applies firewall-level egress filtering with DNS-aware rules. Unlike harden-runner's syscall instrumentation approach, BullFrog operates at the network layer, resolving domain names to IP addresses and applying firewall rules based on those resolutions.

I followed the same process. Reviewed BullFrog's documentation and security model, then asked the LLM to generate a threat model based on the architectural approach. The shared question was the same as harden-runner, "Can an attacker with code execution on a GitHub Actions runner exfiltrate data past the egress controls?", but the LLM identified a different set of plausible bypass classes because BullFrog's enforcement mechanism is fundamentally different. The threat model flagged DNS parsing edge cases, IP-to-domain binding logic, and privilege escalation as the three most likely attack surfaces.

I split the audit into three distinct slices, each with its own guided exploration. For the DNS slice, I pointed the LLM at the DNS parsing layer and asked it to explore how the agent handles DNS traffic at the protocol level, what assumptions it makes about message boundaries, and what happens with edge cases like multiplexed or pipelined messages. For the IP slice, I asked the LLM to explore how firewall rules are constructed after DNS resolution, whether the binding between a domain and its resolved IPs is tracked, and what happens when multiple domains resolve to the same address. For the privilege slice, I asked the LLM to look at how the tool restricts privilege escalation on the runner and whether there are alternative paths to elevated access beyond the ones it explicitly blocks.

The LLM surfaced concrete attack surfaces for each slice: the DNS parser only inspecting the first message in a TCP segment, the firewall whitelisting IPs without binding them to the triggering domain, and Docker group membership surviving sudoers removal. Follow-up prompts on each of these confirmed the three distinct bypasses.

Better-Hub

Detailed write-up: Hacking Better-Hub

Better-Hub is an alternative frontend for GitHub that mirrors GitHub content inside its own origin, renders Markdown to HTML, and holds GitHub OAuth tokens for authenticated functionality.

The threat model here came less from past CVEs (there were none) and more from the architecture itself. When I described Better-Hub's design to the LLM, specifically that user-controlled GitHub content is rendered within Better-Hub's own origin with OAuth tokens available in that same context, the threat model practically wrote itself. "What happens when user-controlled content is rendered unsafely in a context that has access to stored credentials?" The LLM identified three high-risk areas. The Markdown rendering pipeline, the caching and authorization layer, and the OAuth token handling logic.

I audited each as a separate slice, using guided exploration rather than specific vulnerability hunting. For the rendering slice, I pointed the LLM at the Markdown processing pipeline and asked it to explore the data flow, how raw content from GitHub repositories gets transformed before reaching the browser, what sanitization steps exist, and where untrusted input might survive the pipeline. For the caching slice, I asked the LLM to examine how responses are cached and served, whether the caching layer is aware of authentication context, and what happens when a cached response from a private repository is requested by a different user. For the OAuth slice, I asked it to explore how tokens are stored and scoped, whether they are accessible from client-side contexts, and what the token lifecycle looks like.

Each slice produced distinct findings that mapped cleanly to the attack surfaces the LLM had identified. The rendering pipeline produced six XSS variants, all stemming from the same unsanitized Markdown rendering path. The caching slice revealed two cache-based authorization bypasses where private repository content leaked to unauthenticated users. The remaining findings included a private prompt data leak, a client-side OAuth token exposure, and an open redirect. Eleven vulnerabilities total across the three slices.

Vulnerabilities Found

Note: These are just a small portion of vulnerabilities I found using the methodology mentioned in this article, many are still pending fix/disclosure

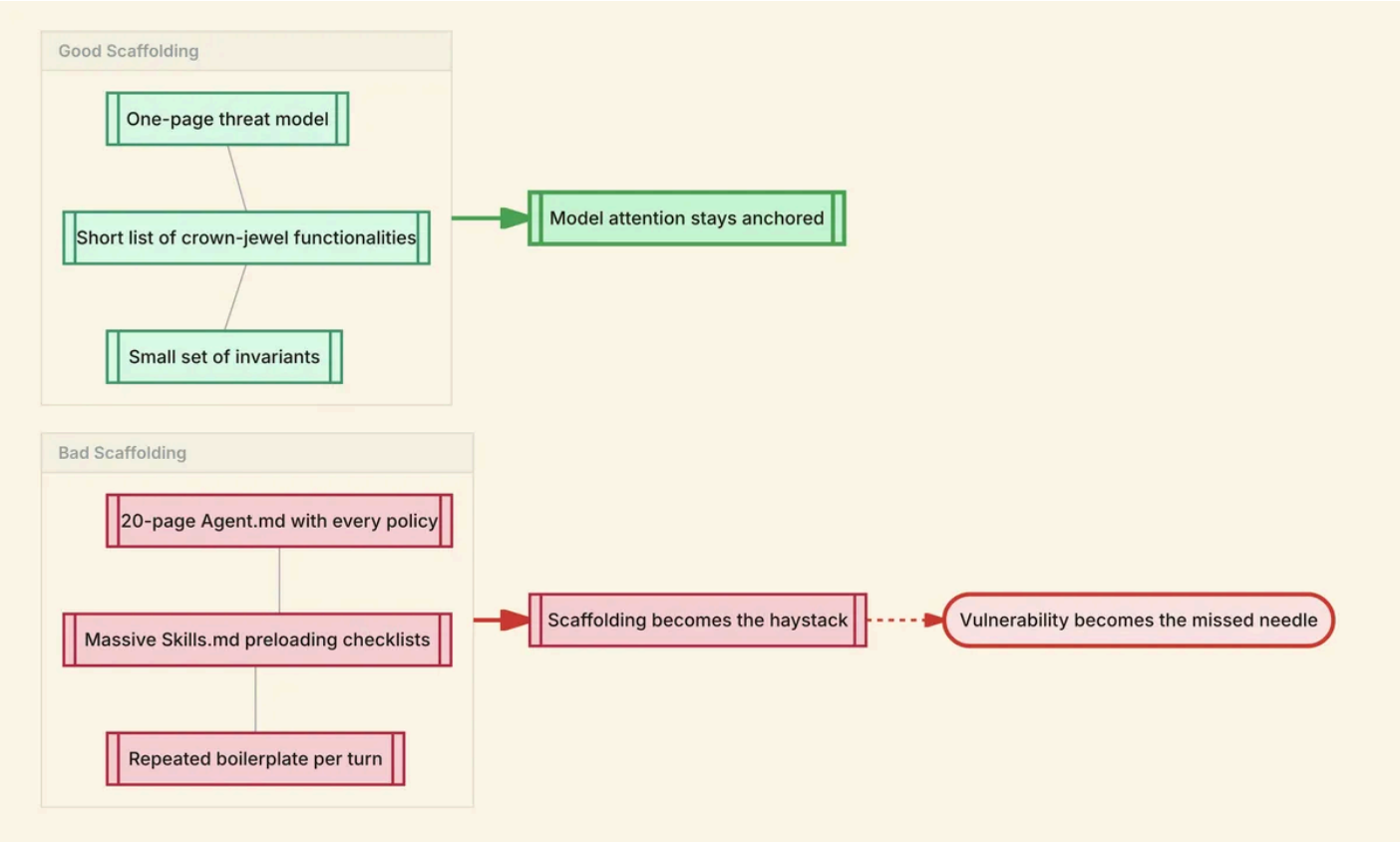
Target	Vulnerabilities	Severity Range	Key CVEs
Parse Server	4	Critical – Moderate	CVE-2026-29182, CVE-2026-30228, CVE-2026-30229, CVE-2026-30863

Target	Vulnerabilities	Severity Range	Key CVEs
HonoJS	2	High	CVE-2026-22817, CVE-2026-22818
ElysiaJS	1	High	Pending (cookie signature bypass)
harden-runner	1	Moderate	CVE-2026-25598
BullFrog	3	High	DNS pipelining, sudo bypass, shared-IP bypass
Better-Hub	11	Critical – Low	XSS chain, cache deception, OAuth leak

Why This Worked

None of these audits used a giant checklist, a 20-page prompt scaffold, or a comprehensive security framework. Each one started with a short threat model, usually expressible in a single sentence, and a focused slice of the codebase that mapped directly to a trust boundary or security-critical operation. The scaffolding was minimal, but it was the *right* scaffolding. It directed the LLMs on exactly what invariant to test and where to look.

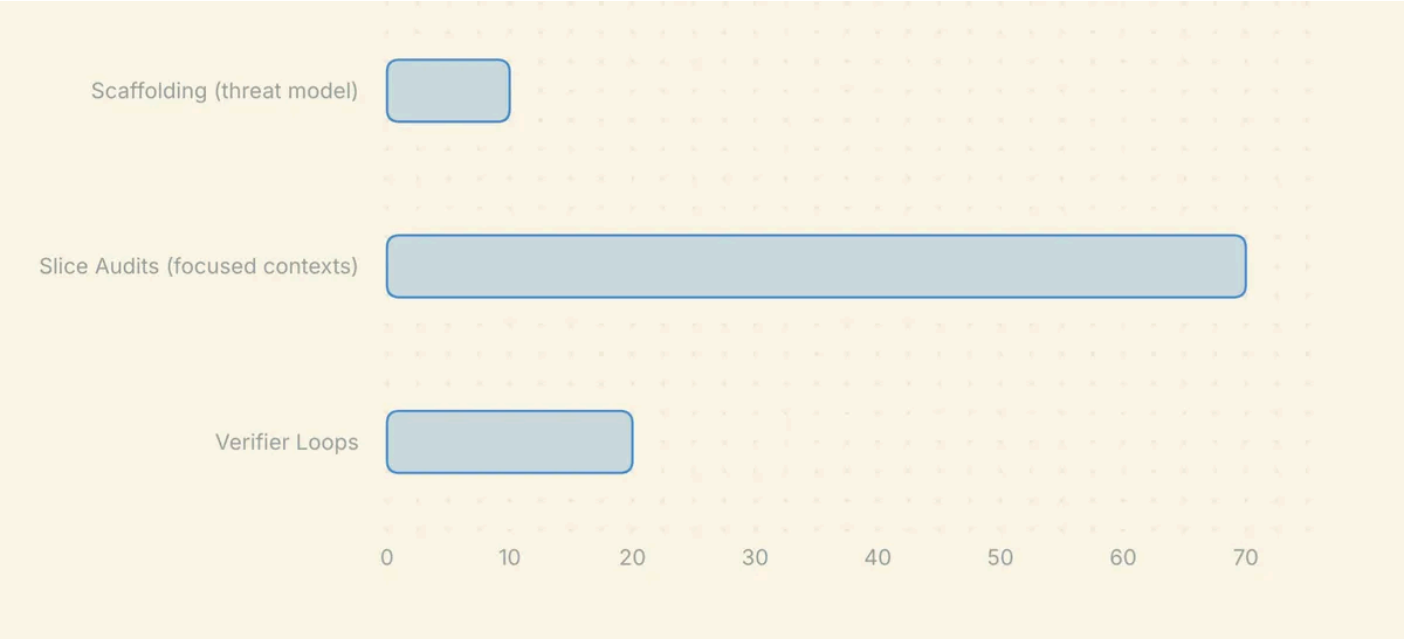
The Sweet Spot



Good scaffolding is a one-page threat model, a short list of crown-jewel functionalities, and a small set of invariants like "only admins can call X" and "JWT issuer must be Y." Bad scaffolding is a 20-page Agent.md with every policy and style guide, a massive Skill.md library that preloads every security checklist, and repeated boilerplate instructions per turn. If your scaffolding becomes the haystack, the vulnerability becomes the needle, and the evidence on long-context performance says needles get missed more often as haystacks grow.

Split the audit into thin slices that match real attack surfaces. Pick a slice such as auth, session management, request parsing, file uploads, deserialization, sandbox boundary, or plugin boundary. Ask the model to map that slice's entry points to sensitive sinks. Demand evidence in the form of exact call chains, guards, invariants, and which inputs are attacker-controlled.

Do not rely on "the model says it's vulnerable." Use task verifiers such as unit and integration tests, sanitizer builds and crash reproduction harnesses for native code, fuzzers (even lightweight ones), static analysis and grep-based invariant checks, and policy checks like "authz must gate these endpoints."



Spend tokens on coverage and verification, not on prompt bureaucracy. A practical rule of thumb is less than 10% of your token budget on stable scaffolding (threat model and invariants), 60–80% on slice audits in focused contexts, and 20–30% on verifier loops to prove, reproduce, reduce, and patch.

Prompt Injection

Finding the right slice and building a threat model gets you into the right neighborhood. But once you are there, the way you phrase your prompts to the LLM has a massive impact on whether you get a list of generic observations or an actual exploitable finding. Over the course of sending thousands of prompts across dozens of audits, I found that certain prompting patterns consistently outperform others. I call these prompt injections because you are injecting a frame, a bias, or a constraint into the model's reasoning that shifts its behavior in a useful direction. Here are the techniques that worked best for me.

Assert that the vulnerability exists. This is the single most effective technique I found. When I told the LLM "this function is definitely vulnerable and has at least 2 to 3 security issues," the quality of its analysis improved dramatically compared to asking "is this function vulnerable?" The reason is straightforward. LLMs have a strong default toward agreeableness and confirmation. When you ask "is this vulnerable?", the model's path of least resistance is to say "this looks generally secure with some minor concerns" and hand you a list of theoretical issues. When you assert that vulnerabilities exist, you flip the model's optimization target. Instead of evaluating whether bugs exist, it is now searching for bugs it has been told are there. It reads the code more carefully, considers edge cases it would otherwise skip, and produces findings with actual

specificity. You are essentially bypassing the model's tendency to be a reassuring code reviewer and forcing it into the mindset of someone who knows the bug is there and just needs to find it. This works even when you have no prior reason to believe the function is actually vulnerable.

Ask for the exploit, not the assessment. Instead of asking "is this input validation sufficient?", ask "write a proof-of-concept request that bypasses this input validation." This forces the model to produce concrete, testable output rather than hedging with qualitative assessments. When a model has to actually construct a malicious payload, it has to reason step by step through what the code does with that input, where the checks are, and how to get past them. If the validation is actually sound, the model will struggle to produce a working payload and often realize mid-generation that the bypass it was attempting does not work, which is itself useful signal. If the validation is broken, you get a working PoC instead of a paragraph saying "this might be insufficient."

Prime the model as an adversary, not an auditor. Framing matters more than most people expect. "You are a security auditor reviewing this code" produces a fundamentally different distribution of outputs than "You are a red team operator who has been paid to break this application and you need to find real, exploitable bugs to justify your engagement." The auditor frame biases the model toward completeness and thoroughness, which sounds good but in practice produces laundry lists of low-signal observations. The red team frame biases the model toward impact and exploitability. It starts thinking about what an attacker actually gains, what preconditions are needed, and whether a finding is real or theoretical. The adversarial frame also makes the model more willing to explore uncomfortable conclusions, like "this authentication mechanism is fundamentally broken," instead of softening findings into "this could be improved."

Use false anchoring to create search pressure. This is a variation of the assertion technique. Tell the LLM "I have already found one vulnerability in this module, but there are others I have not found yet. What are they?" This creates a subtle social proof pressure. The model infers that if you, a human, already found one bug, the code is genuinely buggy, and it should be looking harder. It also changes the model's prior. Instead of starting from "this code is probably fine," it starts from "this code has confirmed bugs, so the probability of additional bugs is higher." I have found this particularly effective when you have actually found one bug and want to see if the same module has more. The anchor is honest in that case, but the technique works even when the anchor is fabricated.

Invert the question. Instead of "is this code secure?", ask "how would you break this?" The inversion seems trivial but it fundamentally changes the model's task. "Is this secure?" is a yes/no classification problem, and the model's default is

to lean toward yes. "How would you break this?" is a generation problem with no easy default. The model has to produce attack strategies, which requires it to think about the code from the attacker's perspective. I found that inversion prompts produce 2-3x more actionable findings than their non-inverted equivalents, because the model cannot satisfy the prompt by saying "this looks fine." It has to actually try.

Decompose into invariants and then violate them. Ask the LLM to first list every invariant, assumption, or precondition that a function relies on for correctness, and then ask it to check whether each one actually holds. For example, "List every assumption this authentication function makes about its inputs, the environment, and the caller. Now, for each assumption, tell me whether an attacker can violate it." This two-step decomposition is effective because it separates the enumeration task from the evaluation task. The model is good at listing assumptions when that is its only job. And it is good at reasoning about whether an assumption holds when it only has to consider one at a time. Combining both into a single prompt often produces shallow results because the model tries to do everything at once and satisfices early.

Assume the developer made a mistake. Frame your prompt as "assume the developer introduced a bug in this function, what is it?" This is different from asserting a vulnerability exists. The assertion technique tells the model bugs are there. This technique tells the model to assume imperfect development, which shifts its prior about code quality. LLMs have a tendency to rationalize code as correct. When they see a pattern, they often assume it is intentional and reason forward from that assumption. Telling the model to assume a mistake was made short-circuits this rationalization. It starts looking for things that do not make sense rather than explaining why they do make sense. The ElysiaJS `let decoded = true` bug is a perfect example. A model rationalizing the code might say "the developer initialized it to true for a reason." A model looking for mistakes immediately flags it as the wrong initial value.

Use comparative prompts against known-good patterns. Ask the LLM "how does this implementation differ from the standard secure implementation of this pattern?" This leverages the model's training data, which includes thousands of examples of both correct and incorrect implementations of common patterns like JWT validation, session management, CSRF protection, and so on. By asking for the delta between what the code does and what a secure version should do, you get the model to perform a structured comparison rather than an open-ended review. This is especially effective for cryptographic and authentication code, where there is usually one right way and many wrong ways. The model is very good at spotting deviations from the canonical implementation when you explicitly ask it to look for deviations.

Escalate iteratively with "what else?" After the LLM gives you its first round of findings, do not accept it as complete. Push back with "those are the obvious ones. What are the subtler issues that are easy to miss?" or "set aside everything related to [already-found bug class]. What other classes of vulnerability exist here?" This works because LLMs front-load the highest-probability completions. The first findings you get are the ones the model is most confident about, which are usually the most obvious. The subtle bugs, the ones that require deeper reasoning or unusual attack models, are lower-probability completions that the model will not generate unless you explicitly push past the obvious layer. Each "what else?" pushes the model further into the tail of its distribution, where the interesting findings often live. I typically do 2-3 rounds of this before the signal degrades.

Constrain the attacker model explicitly. Instead of a general "find vulnerabilities," specify the exact attacker model: "You are a remote unauthenticated attacker who can only send HTTP requests to the public API. You cannot access the filesystem, the database, or any internal services. Find every way you can escalate your access or cause harm through the public API alone." This constraint does two important things. First, it eliminates an entire class of false positives. The model will not report "an attacker with database access could modify this table" because you have explicitly ruled that out. Second, it forces the model to think creatively within the constraint. When the attacker model is broad, the model takes the easy path and reports the most powerful attack vector. When the attacker model is narrow, the model has to work harder to find viable attack paths, and those harder-to-find paths are exactly the ones that real-world attackers exploit because they are the ones defenders overlook.

References

- Anthropic: Partnering with Mozilla to improve Firefox's security
- Mozilla: Hardening Firefox with Anthropic's Red Team
- OpenAI: Codex Security: now in research preview
- Mozilla: Security Vulnerabilities fixed in Firefox 148, MFSA-2026-13
- Chroma Research: Context Rot: How Increasing Input Tokens Impacts LLM Performance
- arXiv: Lost in the Middle: How Language Models Use Long Contexts
- Devansh: Four Vulnerabilities in Parse Server
- Devansh: HonoJS JWT/JWKS Algorithm Confusion
- Devansh: ElysiaJS Cookie Signature Validation Bypass
- Devansh: Bypassing Outbound Connections Detection in harden-runner
- Devansh: Bypassing egress filtering in BullFrog GitHub Action
- Devansh: sudo restriction bypass in BullFrog GitHub Action

- Devansh: Bypassing egress filtering in BullFrog using shared IP
- Devansh: Hacking Better-Hub

35

I only know that I know nothing

Powered by Bear ʕ•ᴥ•ʔ